HÀM (FUNCTION)

> **Bám sát nội dung về** Hàm: def, tham số, return, tham số linh hoạt, phạm vi biến, lambda, đệ quy, module hóa

Mục tiêu sau buổi thực hành

- Viết và gọi được hàm bằng def, dùng return để trả về một hoặc nhiều giá trị.
- Sử dụng thành thạo tham số mặc định, tham số từ khóa, *args, **kwargs.
- Phân biệt được biến local/global, biết dùng từ khóa global khi cần.
- Viết được hàm lambda, kết hợp với map(), filter(), sorted().
- Hiểu và viết được hàm đệ quy (giai thừa, Fibonacci); tổ chức code thành module riêng.

*🔑 **Lưu ý:** Từ buổi này, def đã chính thức được học nên mọi bài tập được phép - và nên - đóng gói logic thành hàm riêng thay vị viết lặp lại trực tiếp như các buổi trước.*

Hoạt động 1: Hàm cơ bản – def, tham số, return

Bài tập 1.1 – Viết hàm và gọi lại nhiều lần: Viết các hàm sau, mỗi hàm gọi thử với ít nhất 3 bộ dữ liệu khác nhau:

```
def uscln(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def bscnn(a, b):
    return a * b // uscln(a, b)

def kiem_tra_nguyen_to(n):
    if n < 2:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

def kiem_tra_so_hoan_thien(n):
    tong_uoc = 0
    for i in range(1, n):
        if n % i == 0:
            tong_uoc += i
    return tong_uoc == n

print(uscln(24, 36))
print(bscnn(4, 6))
```

MẠNH ĐẠT

```
print(kiem_tra_nguyen_to(29))
print(kiem_tra_so_hoan_thien(28))   # 28 = 1 + 2 + 4 + 7 + 14
```

Bài tập 1.2 – return không giá trị và trả về nhiều giá trị:

```
def in_loi_chao(ten):
    print(f"Xin chao, {ten}!")
    return   # ham khong tra ve gia tri (tra ve None)

def chia_lay_thuong_du(a, b):
    return a // b, a % b   # tra ve nhieu gia tri qua tuple

in_loi_chao("An")
thuong, du = chia_lay_thuong_du(17, 5)
print(f"Thuong: {thuong}, du: {du}")
```

Hoạt động 2: Tham số mặc định & tham số từ khóa

```
def gioi_thieu(ten, tuoi=18, lop="Chua ro"):
    print(f"Ten: {ten} - Tuoi: {tuoi} - Lop: {lop}")

gioi_thieu("An")                              # dung het gia tri mac dinh
gioi_thieu("Binh", 20)                         # ghi de tuoi
gioi_thieu("Chi", lop="CNTT01")                # dung tham so tu khoa, bo qua tuoi
gioi_thieu(ten="Dung", lop="CNTT02", tuoi=19)  # thu tu tham so tu khoa co the dao lon
```

❓ **Yêu cầu:** *Giải thích vì sao khi gọi hàm bằng tham số từ khóa (ten=..., lop=...) thì thứ tự truyền vào không quan trọng.*

Hoạt động 3: Tham số linh hoạt – *args và **kwargs

Bài tập 3.1 – *args: tính tổng số lượng bất kỳ các số:

```
def tinh_tong(*args):
    tong = 0
    for so in args:
        tong += so
    return tong

print(tinh_tong(1, 2, 3))
print(tinh_tong(5, 10, 15, 20, 25))
print(tinh_tong())   # khong truyen so nao -> tra ve 0
```

Bài tập 3.2 – **kwargs: in thông tin động:

```
def in_thong_tin(ho_ten, tuoi, **kwargs):
```

```
    print(f"Ho ten: {ho_ten} - Tuoi: {tuoi}")
    for khoa, gia_tri in kwargs.items():
        print(f"  {khoa}: {gia_tri}")

in_thong_tin("Nguyen Van A", 20, lop="CNTT01", que_quan="Ha Noi")
in_thong_tin("Tran Thi B", 21, email="b@example.com")
```

Hoạt động 4: Phạm vi biến – local, global, từ khóa global

```
so_luot_truy_cap = 0   # bien global

def tang_luot_truy_cap():
    global so_luot_truy_cap
    so_luot_truy_cap += 1

def vi_du_bien_local():
    so_luot_truy_cap = 100   # day la bien LOCAL, khac voi bien global cung ten
    print("Ben trong ham, bien local =", so_luot_truy_cap)

tang_luot_truy_cap()
tang_luot_truy_cap()
print("So luot truy cap (global):", so_luot_truy_cap)

vi_du_bien_local()
print("Sau khi goi ham, bien global van la:", so_luot_truy_cap)
```

❓ **Yêu cầu:** *Giải thích vì sao nếu bỏ dòng global so_luot_truy_cap trong hàm tang_luot_truy_cap(), chương trình sẽ báo lỗi UnboundLocalError.*

Hoạt động 5: Hàm lambda kết hợp map(), filter(), sorted()

Bài tập 5.1 – map() với lambda:

```
danh_sach_so = [1, 2, 3, 4, 5]
binh_phuong = list(map(lambda x: x ** 2, danh_sach_so))
print(binh_phuong)
```

Bài tập 5.2 – filter() với lambda:

```
so_chan = list(filter(lambda x: x % 2 == 0, danh_sach_so))
print(so_chan)
```

Bài tập 5.3 – sorted() với lambda: sắp xếp danh sách sinh viên theo điểm:

```
danh_sach_sv = [
    {"ten": "An", "diem": 8.5},
```

```
    {"ten": "Binh", "diem": 7.0},
    {"ten": "Chi", "diem": 9.2},
]

sap_xep_theo_diem = sorted(danh_sach_sv, key=lambda sv: sv["diem"])
sap_xep_giam_dan = sorted(danh_sach_sv, key=lambda sv: sv["diem"], reverse=True)

for sv in sap_xep_theo_diem:
    print(sv["ten"], "-", sv["diem"])

print("--- Giam dan ---")
for sv in sap_xep_giam_dan:
    print(sv["ten"], "-", sv["diem"])
```

❓ **Yêu cầu:** *So sánh cách sắp xếp này với cách "đặt điểm trước tên trong tuple" đã dùng ở Buổi 3 — vì sao dùng key=lambda linh hoạt hơn?*

Hoạt động 6: Đệ quy – Giai thừa, Fibonacci

Bài tập 6.1 – Giai thừa bằng đệ quy

❓ **Yêu cầu:** Sau khi thực hiện bằng đệ quy, hãy **so sánh với vòng lặp** việc sử dụng vòng lặp

```
def giai_thua_de_quy(n):
    if n <= 1:          # dieu kien dung
        return 1
    return n * giai_thua_de_quy(n - 1)

def giai_thua_lap(n):
    ket_qua = 1
    for i in range(1, n + 1):
        ket_qua *= i
    return ket_qua

print(giai_thua_de_quy(5), "-", giai_thua_lap(5))
```

Bài tập 6.2 – Số Fibonacci thứ n bằng đệ quy:

```
def fibonacci_de_quy(n):
    if n <= 1:          # dieu kien dung
        return n
    return fibonacci_de_quy(n - 1) + fibonacci_de_quy(n - 2)

for i in range(10):
    print(fibonacci_de_quy(i), end=" ")
print()
```

❓ **Yêu cầu:** *Thử tính fibonacci_de_quy(30), quan sát thời gian chạy chậm hơn hẳn so với giai_thua_de_quy(30), trả lời vì sao đệ quy Fibonacci "tốn kém" hơn (gợi ý: số lần gọi hàm tăng theo cấp số nhân do tính lại nhiều lần các giá trị trùng nhau).*

Hoạt động 7: Mini project – Module hóa chương trình

Tạo 2 file trong cùng thư mục: **`utils.py`** (module chứa các hàm xử lý chuỗi và số đã học) và **`main.py`** (chương trình chính, import và sử dụng module).

File utils.py:

```
def dao_nguoc_chuoi(chuoi):
    return chuoi[::-1]

def kiem_tra_palindrome(chuoi):
    return chuoi == chuoi[::-1]

def chuan_hoa_ho_ten(chuoi):
    return " ".join(chuoi.split()).title()

def uscln(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def kiem_tra_nguyen_to(n):
    if n < 2:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

File main.py:

```
import utils

print(utils.dao_nguoc_chuoi("Python"))
print(utils.kiem_tra_palindrome("madam"))
print(utils.chuan_hoa_ho_ten("   nguyen   van   an  "))
print(utils.uscln(24, 36))
print(utils.kiem_tra_nguyen_to(29))
```

Chạy chương trình bằng lệnh Terminal / Command Line:

```
python main.py
```

❓ **Yêu cầu:** *Giải thích vì sao utils.py và main.py cần đặt trong cùng một thư mục để lệnh import utils hoạt động đúng.*

Sản phẩm nộp cuối buổi & Tiêu chí đánh giá

📂 **Danh sách file sản phẩm nộp:**

- File hoat_dong_1_2_3.py (hàm cơ bản, tham số mặc định, *args/**kwargs).
- File hoat_dong_4_5.py (phạm vi biến, lambda kết hợp map/filter/sorted).
- File hoat_dong_6.py (đệ quy giai thừa, Fibonacci).
- Cặp file utils.py và main.py (mini project module hóa).

- Chụp ảnh thực thi thành công các hoạt động
- Trả lời các nội dung đã yêu cầu.
- Link Github
- Đặt trong file PDF để nộp bài

TRƯƠNG MẠNH ĐẠT